



1+2

16

1920 sp

عملي

كلية الهندسة المعلوماتية

السنة الثالثة

Introduction & Drawing primitives

عمل طلابي



RB Informatics;

محتوى مجاني غير مخصص للبيع التجاري
3/11/2024

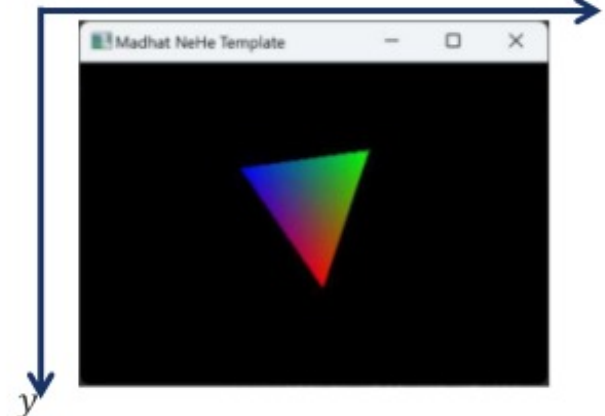
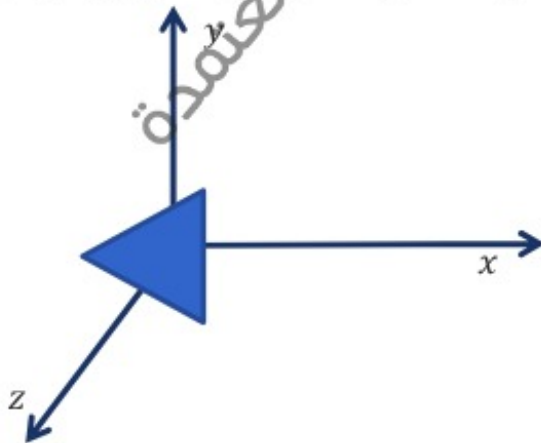
بيانات حاسوبية

مرحباً بكم زملائنا الأعزاء في مقر العمل لمادة البيانات الحاسوبية، نريد التنويه أن محتوى محاضراتنا للقسم العملي سيكون معتمداً على OpenGL legacy فقط، كما سيعرض ملخصاً للأفكار المعروضة في الكورس المذكور أدناه إضافة لملفات محاضرات الكلية، بحيث أن المقرر سيحتوي جميع الأفكار المطروحة من قبل معيدي المادة ولكن سنعتمد على الكورس والملفات في شرحنا لهذه المفاهيم والأفكار. الكورس موجود على Youtube بعنوان: دورة أوبن جي إل نوار المكيس وأيهم طالب

OpenGL (Open Graphics Library)

OpenGL (Open Graphics Library) هي واجهة برمجة التطبيقات مستقلة المنصة (تعمل في عدة بيئات وعدة أنظمة تشغيل) مصممة لإنتاج رسومات الحاسب ثنائية الأبعاد وثلاثية الأبعاد، ومتوافقة مع عدة لغات برمجية وسنستخدمها في لغة C++. تستخدم لرسم مشاهد ثلاثية الأبعاد ومحاكاة بيئات افتراضية، وتعتمد على عناصر رسومية بسيطة مثل النقاط والخطوط والمضلعات. تعتبر OpenGL أداة قياسية وفعالة في تطوير ألعاب مثل counter-strike و call of Duty.

في OpenGL، العالم الذي نتعامل معه هو ثلاثي الأبعاد (3D)، لكن العرض النهائي على الشاشة يكون ثنائي الأبعاد (2D) لأن الشاشة مسطحة. لتحويل المجسمات 3D إلى عرض 2D، تستخدم OpenGL تقنيات مثل الإسقاط المنظوري (perspective projection) حيث تعتمد على معادلات رياضية موجودة في التوابع داخل المكتبات لتحويل الإحداثيات ثلاثية الأبعاد إلى إحداثيات ثنائية الأبعاد تعرض على الشاشة. هذه العمليات تضمن الحفاظ على العمق والواقعية في المشهد النهائي.



مركز العالم في OpenGL:

عادةً ما يكون مركز العالم الافتراضي هو نقطة الأصل (0,0,0) حيث تتقاطع المحاور الثلاثة. هذا المركز هو النقطة المرجعية التي يتم قياس باقي الإحداثيات بالنسبة إليها. عند إنشاء الأجسام أو المجسمات في OpenGL ، نقوم بتحديد إحداثيات الرؤوس (vertices) بالنسبة إلى هذا المركز.

التعامل مع التوابيع في OpenGL:

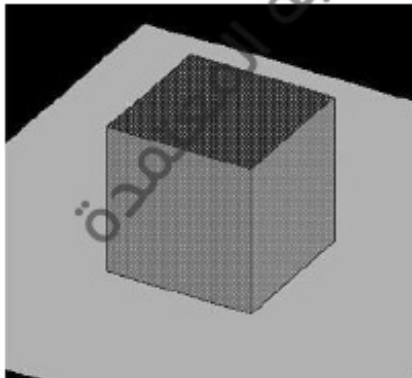
OpenGL توفر العديد من الدوال الجاهزة التي تساعد في عمليات الإسقاط (projection) وتحويل الإحداثيات من الفراغ ثلاثي الأبعاد إلى شاشة ثنائية الأبعاد دون الحاجة إلى كتابة المعادلات الرياضية يدوياً. في عالم OpenGL ، المحور z له دور مهم في تحديد مواضع الأجسام للمراقب:

- قبل المراقب (المحور z السالب): الأجسام التي تكون إحداثياتها على المحور z سالبة تكون أمام المراقب، وبالتالي تكون مرئية على الشاشة. هذه هي الأجسام التي تقع في مجال الرؤية لدينا.
- خلف المراقب (المحور z الموجب): الأجسام التي تكون إحداثياتها على المحور z موجبة تقع خلف المراقب، وبالتالي تكون محجوبة عن الرؤية لأنها خارج نطاق مجال رؤيتنا. بمعنى آخر، لا يتم عرض هذه الأجسام لأنها تقع خلف الكاميرا الافتراضية.

مراحل تحويل المجسمات إلى مشاهد واقعية:

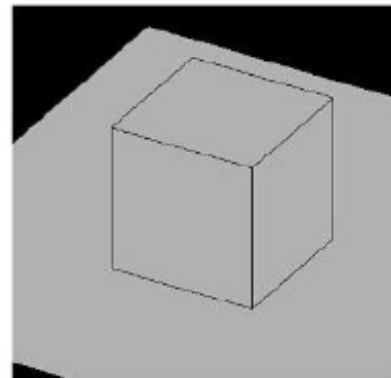
2. color:

نبدأ برسم الهيكل الأساسي للشكل (مثل المكعب) باستخدام رؤوس (نقاط) وحواف (خطوط)، لكن دون إضافة أي واقعية.



1. Perspective:

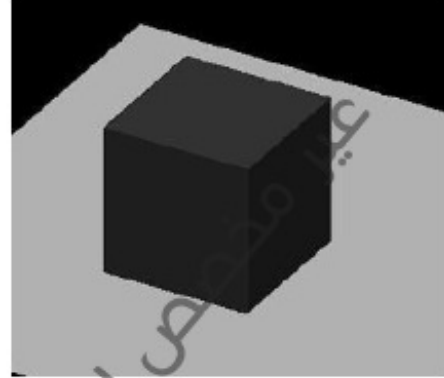
نبدأ برسم الهيكل الأساسي للشكل (مثل المكعب) باستخدام رؤوس (نقاط) وحواف (خطوط)، لكن دون إضافة أي واقعية.





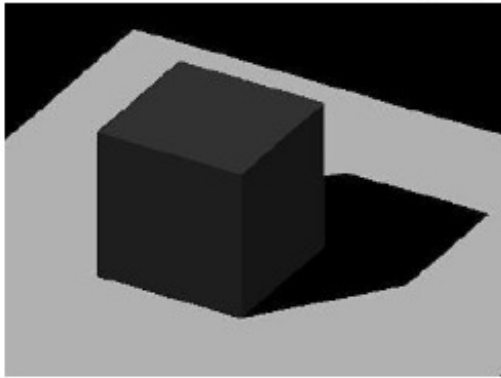
3. Lighting:

عند إضافة الإضاءة، تختلف درجة السطوع بين الأوجه، مما يعزز من عمق الشكل.



4. Shadow:

بعد إضافة الإضاءة، يبدأ الجسم بحجب الضوء عن الأجسام المحيطة، مما ينتج عنه الظلال التي تعزز من الشعور بالعمق والواقعية.



5. Texture Mapping:

بدلاً من تقسيم الوجه إلى مثلثات صغيرة جداً (التي تستهلك ذاكرة ومعالجة)، نستخدم تقنية (Texture Mapping) لوضع صورة مفصلة على سطح الجسم، مثل رسم تفاصيل الخشب على الصندوق.



6. Aliasing and Anti-aliasing:

لمعالجة التشويش الناتج عن البكسلات، نستخدم تقنيات Anti-aliasing لتنعيم حواف الصورة.

7. fog:

يضيف الضباب إحساساً بالمسافة البعيدة من خلال جعل التفاصيل أقل وضوحاً في الأطراف.

8. Blend and Transparency:

نستخدم الشفافية لمحاكاة المواد مثل الزجاج، حيث يمكن رؤية الأجسام خلفه لكن بشكل غير واضح، اعتماداً على درجة الشفافية.



تجري الرياح كما تجري سفينتنا نحن الرياح ونحن البحر والسفن
إن الذي يرتجي شيئاً بهمته يلقاه لو حاربه الإنسان والجن!
فاقصِد إلى قمم الأشياء تدرّكها تجري الرياح كما رادت لها السفن

OpenGL Data Types

OpenGL Data Type	Internal Representation	Defined as C Type	C Literal suffix
GLbyte	8-bit integer	Signed char	b
GLshort	16-bit integer	Short	s
GLint, GLsizei	32-bit integer	Long	l
GLfloat, GLclampf	32-bit floating point	Float	f
GLdouble, GLclampd	64-bit floating point	Double	d
GLboolean, GLubyte	8-bit unsigned integer	Unsigned char	ub
GLushort	16-bit unsigned integer	Unsigned short	us
GLuint, GLenum, GLbitfield	32-bit unsigned integer	Unsigned long	ul
GLchar	8-bit character	char	None
GLintptr, GLsizeiptr	Native pointer	ptrdiff_t	None

ملاحظة: لماذا يوجد data types مختلفة لمكتبة OpenGL؟

لأنها مكتبة متوافقة مع عدة لغات برمجية ونظراً لاختلاف طرق تسمية المتغيرات بين لغات البرمجة (مثلاً في c++ نكتب bool بينما في java نكتب Boolean) تم إنشاء متغيرات خاصة بـ OpenGL.

فهم التوابع وتسمية دوال OpenGL:

يتألف اسم كل دالة في OpenGL من أربعة أقسام رئيسية، والتي تحدد نوع البيانات المرسل إلى هذه الدالة. دعونا نفصل هذه الأقسام:

prefix

Function Name

Number of parameters

Type Suffix

المقطع الأول (prefix): يشير إلى المكتبة أو الفئة التي تنتمي إليها الدالة، وعادة ما تكون "gl" لدوال OpenGL.

المقطع الثاني (function name): يصف وظيفة الدالة، مثل "color" للدوال المتعلقة بالألوان.

المقطع الثالث (Number of parameters): يشير إلى عدد المعاملات التي تأخذها الدالة. على سبيل المثال، "3" في glColor3f يعني أن الدالة تأخذ ثلاثة معاملات.

المقطع الرابع (Type Suffix): يحدد نوع البيانات التي يتم استخدامها في المعاملات. على سبيل المثال، "f" في glColor3f يشير إلى أن المعاملات هي من نوع float (عدد عشري)، ويمكن معرفة الـ suffix المناسبة لكل نوع من البيانات من الجدول الذي ذكرناه سابقاً.

في الـ template المبني بواسطة jeff Molofee (يمكنكم الوصول إليه من خلال مسح الـ QR code المذكور في نهاية ملحق تنصيب البيئة ونسخ ملف main.cpp إلى مشروعكم) يظهر لدينا التابعين التاليين:

1. DrawGLScene:

يستخدم هذا التابع لرسم المشهد على الشاشة. يتم استدعاؤه بشكل متكرر لتحديث الرسومات بمعدل 24 مرة بالثانية. حيث يقوم بمسح الشاشة وإعادة رسم كل الكائنات في كل frame. **ملاحظة:** عند تجريب أي كود يوضع في هذا التابع حصراً ويبقى باقي الـ template كما هو.

2. InitGL:

تابع لتهيئة إعدادات OpenGL مثل الألوان، الإضاءة، والباعدات الأساسية الأخرى التي يجب تجهيزها قبل بدء الرسم، يستدعى لمرة واحدة في بداية البرنامج.

الـ primitives في OpenGL:

الـ Primitives هي أشكال بدائية نستطيع رسمها باستخدام توابع جاهزة في OpenGL، ونستطيع باستخدامها رسم أشكال أكثر تعقيداً.

أمثلة عن الـ Primitives: Points, lines, quadrilaterals, and general polygons.

طريقة الاستخدام:

عندما نريد رسم أحد الـ Primitives يجب وضع الكود الخاص بهذا الـ Primitive ضمن scoop شكله العام:

```
glBegin(MODE);
```

```
// drawing primitives
```

```
glEnd();
```

حيث تعمل التوابع glBegin(MODE) و glEnd() على تحديد بداية ونهاية الكتلة التي يتم فيها رسم الـ Primitive. نمرر في glBegin(MODE) الـ MODE كمعامل والذي يحدد نوع الـ Primitive (مثل GL_POINTS, GL_LINES, GL_TRIANGLES, GL_QUADS, GL_POLYGON).

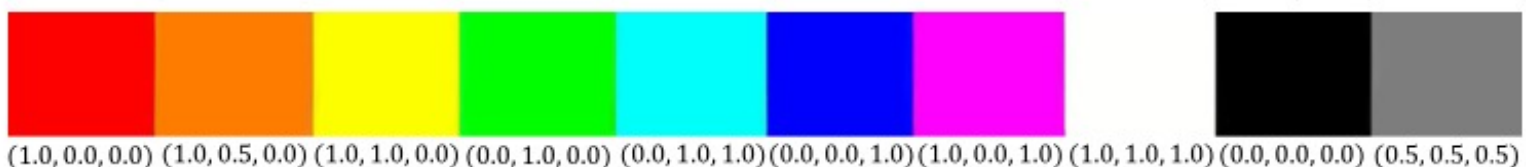
بعد كتابة glEnd() لا يمكننا إضافة المزيد من الـ Primitives التابعة للـ MODE الذي حددناه في الـ glBegin().

تغيير لون الأشكال في OpenGL باستخدام glColor3f:

تستخدم الدالة glColor3f في OpenGL لتحديد اللون الحالي للأشكال (primitives) التي سيتم رسمها. تعمل هذه الدالة على تغيير لون النقاط، الخطوط، والمثلثات وغيرها من الأشكال.

```
glColor3f (red, green, blue);
```

وتستخدم القيم العشرية من 0.0 إلى 1.0 لكل من الألوان (أحمر، أخضر، أزرق).



أما الدالة glColor3ub تستخدم القيم الصحيحة من 0 إلى 255، مما يجعلها أقرب لنظام ألوان الـ RGB المتعارف عليه. glColor3ub(red, green, blue);

ملاحظة: عند تغيير اللون في مرحلة معينة من الكود، يتغير اللون لجميع العناصر التي نضيفها بعد هذه المرحلة، ولإعادته اللون الأساسي نعيد كتابة التعليمة ونغيره مرة أخرى.

أنواع الـ primitives:

1. GL_POINTS:

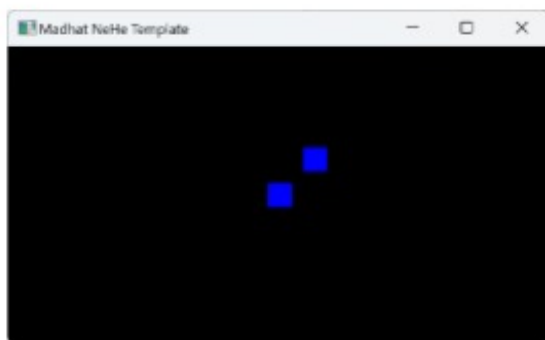
الوصف: تستخدم لرسم نقاط فردية في الفضاء ثلاثي الأبعاد.
الخصائص:

يتم رسم كل نقطة باستخدام التابع glVertex الذي يستقبل إحداثيات ثنائية أو ثلاثية الأبعاد. يمكن التحكم في حجم النقطة باستخدام glPointSize(size)، حيث يكون الحجم الافتراضي بيكسل واحد. لتحسين مظهر النقطة، يمكن استخدام GL_POINT_SMOOTH لجعل النقطة تبدو كدائرة عند تكبير حجمها.

مثال:

```
int DrawGLScene(GLvoid) // Here's Where We Do All The Drawing
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();

    glPointSize(20); // تحديد حجم النقطة ليكون 20 بيكسل
    glBegin(GL_POINTS);
        glColor3f(0.0f, 0.0f, 1.0f);
        // تحديد اللون الأزرق للنقاط
        glVertex3f(0.0f, 0.0f, -1.0f);
        // تحديد الإحداثيات للنقطة الأولى
        glVertex3f(0.1f, 0.1f, -1.0f);
        // تحديد الإحداثيات للنقطة الثانية
    glEnd(); // نهاية رسم النقاط
    return TRUE;
}
```



ملاحظة:

glVertex3f(x, y, z): يُستخدم لرسم نقطة عن طريق تحديد إحداثياتها (x, y, z) ويتم تمريرها كـ float لأن التابع يحوي الـ "f" suffix الخاصة بـ float، كما يوجد أنواع أخرى للتابع مثل glVertex2f(x, y) نمرر به إحداثيين (x, y) فقط من نوع float (ويكون الإحداثي z يساوي 0 بشكل افتراضي) أو glVertex2d(x, y) لتمرير إحداثيات من نوع double أو ... وذلك تبعاً للرقم والحرف المذكورين في نهاية اسم التابع.

أمثلة على رسم أشكال مختلفة باستخدام النقاط:

1. رسم خط مستقيم من نقاط:

الفكرة: في رسم خط مستقيم، نحتاج إلى تحديد سلسلة من النقاط التي تشكل الخط. بدلاً من رسم خط واحد فقط، يمكن استخدام حلقة لرسم مجموعة من النقاط التي تقترب من بعضها البعض لتشكيل الخط.

```
int DrawGLScene(GLvoid)
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    glPointSize(20);
    glBegin(GL_POINTS);
    for (GLfloat i = -1.0; i <= 1.0; i += 0.01)
        glVertex3f (i, 0.0f, -3.0f);
    glEnd();
    return TRUE;
}
```

الشرح:

- 🔥 glEnd() و glBegin(GL_POINTS): تحديد بداية ونهاية رسم النقاط.
- 🔥 for(GLfloat i = -1.0; i <= 1.0; i += 0.01): حلقة تكرارية لرسم نقاط على طول خط مستقيم حيث أن i هو الإحداثي x لكل نقطة.
- 🔥 نزيد عداد الحلقة بمقدار صغير جداً (0.01) لتكون المسافات بين النقاط غير ملحوظة ويظهر كخط غير متقطع في الشاشة.
- 🔥 نستخدم قيم 1.0f للإحداثي y و 1.0f للإحداثي z لجعل النقاط تتواجد على طول خط أفقي في الفضاء.
- 🔥 النقاط يتم رسمها على طول محور x مع تغيير قيمة i، مما يخلق خطاً.

ملاحظة: يجب أن نتجنب هذه الطرق برسم الخطوط لأنها ذات دقة منخفضة كما أن زيادة العناصر يجعل تنفيذ البرنامج أبطأ.

2. رسم دائرة من نقاط:

الفكرة: لرسم دائرة، نحتاج إلى حساب إحداثيات النقاط التي تقع على محيط الدائرة. نستخدم معادلات الدائرة المثلثية لحساب هذه الإحداثيات.

المعادلات المثلثية:

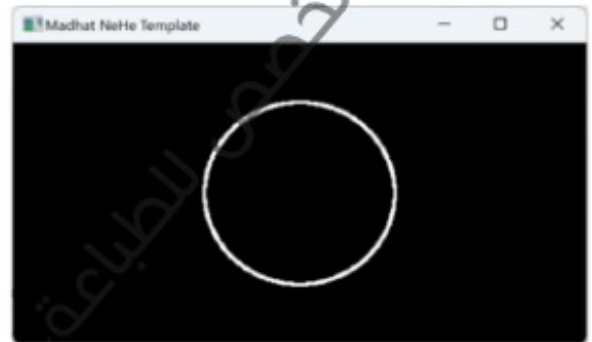
لإحداثيات النقاط على دائرة نستخدم:

$$x = \cos(\text{angle})$$

$$y = \sin(\text{angle})$$

حيث angle هي الزاوية الممثلة من 0 إلى 2π (زاوية كاملة).

```
glPointSize(3);
glBegin(GL_POINTS);
for (GLfloat angle = 0.0; angle <= 2.0 * 3.14f; angle += 0.01f) {
    GLfloat x = cos(angle);
    // حساب الإحداثي x
    GLfloat y = sin(angle);
    // حساب الإحداثي y
    glVertex3f(x, y, -4.0f);
    // رسم النقطة على المحيط
}
glEnd();
```

**الشرح:**

▲ glEnd() و glBegin(GL_POINTS): تحدد بداية ونهاية رسم النقاط.

▲ for(GLfloat angle = 0.0; angle <= 2.0 * 3.14; angle += 0.01f): حلقة تكرارية لرسم نقاط على محيط الدائرة.

▲ Angle: هي الزاوية التي نغيرها من 0 إلى 2π (مع استخدام 3.14 كقيمة تقريبية لـ π).

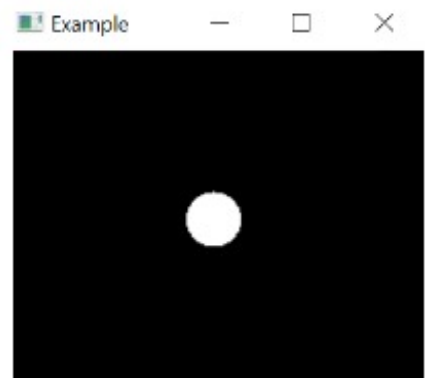
▲ cos(angle) و sin(angle) تعطي الإحداثيات x و y للنقطة على محيط الدائرة.

▲ glVertex3f(x, y, -4.0f): رسم النقطة عند الإحداثيات المحسوبة.

GL_POINT_SMOOTH

هي خاصية في OpenGL تستخدم لجعل النقاط تبدو ناعمة ومدارة بدلاً من أن تظهر كمربعات خشنة. عند تفعيل هذه الخاصية باستخدام glEnable(GL_POINT_SMOOTH)، يتم رسم النقاط بتأثير التنعيم بحيث تظهر كدوائر ناعمة، مما يحسن من مظهر الرسومات عند تكبير النقاط.

```
glEnable(GL_POINT_SMOOTH);
// تفعيل خاصية تنعيم النقاط
glPointSize(30.0f); // تحديد حجم النقطة
glBegin(GL_POINTS);
    glVertex3f(0.0f, 0.0f, -0.1f);
    // رسم نقطة في منتصف الشاشة
glEnd();
```

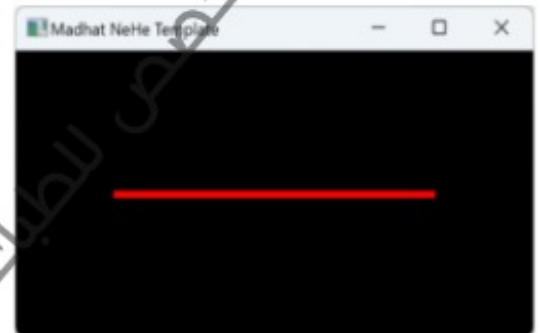


الوصف: تستخدم لرسم خطوط مستقيمة بين كل زوج من النقاط.
الخصائص:

يتطلب رسم نقطتين فأكثر. كل زوج من النقاط يمثل خطاً مستقيماً.
يتم تجاهل النقطة الأخيرة في حال كان عدد النقاط فردياً.
يمكن تغيير عرض الخط باستخدام `glLineWidth(width)`.

مثال:

```
glLineWidth (5);
glBegin(GL_LINES) ;
    glColor3f(1.0f, 0.0f, 0.0f);
    // تحديد اللون الأحمر للخط
    glVertex3f(1.0f, 0.0f, -2.0f);
    glVertex3f(-1.0f, 0.0f, -2.0f);
glEnd();
```



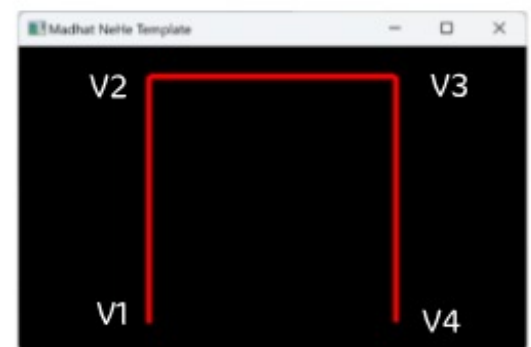
أنماط مختلفة:

1. GL_LINE_STRIP

يرسم خطوط بين كل نقطة والنقطة التالية بشكل متسلسل.
فمثلاً عند تحديد أول نقطتين يقوم برسم خط بينهما، ثم عند إضافة نقطة ثالثة يقوم برسم خط بين الثانية والثالثة وبإضافة نقطة رابعة يضيف خطاً بين الثالثة والرابعة وهكذا...

مثال:

```
glLineWidth(5);
glBegin(GL_LINE_STRIP);
    glColor3f(1.0f, 0.0f, 0.0f); // الأحمر
    glVertex3f(-1.0f, -1.0f, -3.0f);
    glVertex3f(-1.0f, 1.0f, -3.0f);
    glVertex3f(1.0f, 1.0f, -3.0f);
    glVertex3f(1.0f, -1.0f, -3.0f);
glEnd();
```



2. GL_LINE_LOOP :

يشبه GL_LINE_STRIP لكنه في النهاية يقوم بالوصل بين آخر نقطة وأول نقطة لتشكيل حلقة مغلقة.
مثال:

```
glLineWidth(5);
glBegin(GL_LINE_LOOP);
    glColor3f(1.0f, 0.0f, 0.0f); // الأحمر
    glVertex3f(-1.0f, -1.0f, -3.0f);
    glVertex3f(-1.0f, 1.0f, -3.0f);
    glVertex3f(1.0f, 1.0f, -3.0f);
    glVertex3f(1.0f, -1.0f, -3.0f);
glEnd();
```



3. GL_TRIANGLES :

الوصف: تستخدم لرسم مثلثات حيث يتطلب رسم مثلث واحد ثلاث نقاط.
الخصائص:

كل ثلاث نقاط ترسم مثلثاً مستقلاً. يكون الشكل معبئاً من الداخل بشكل افتراضي.
يتم تجاهل النقط الأخيرة في حال كان عدد النقط ليس من مضاعفات الـ 3.

مثال:

```
glBegin(GL_TRIANGLES);
    glColor3f(0.5f, 0.0f, 1.0f); // البنفسجي
    glVertex3f(-1.0f, -1.0f, -3.0f);
    glVertex3f(-1.0f, 1.0f, -3.0f);
    glVertex3f(1.0f, 1.0f, -3.0f);
glEnd();
```



الحياة لن تعطيك كل ما تحب،
لكن القناعة تجعلك تحب كل ما لديك



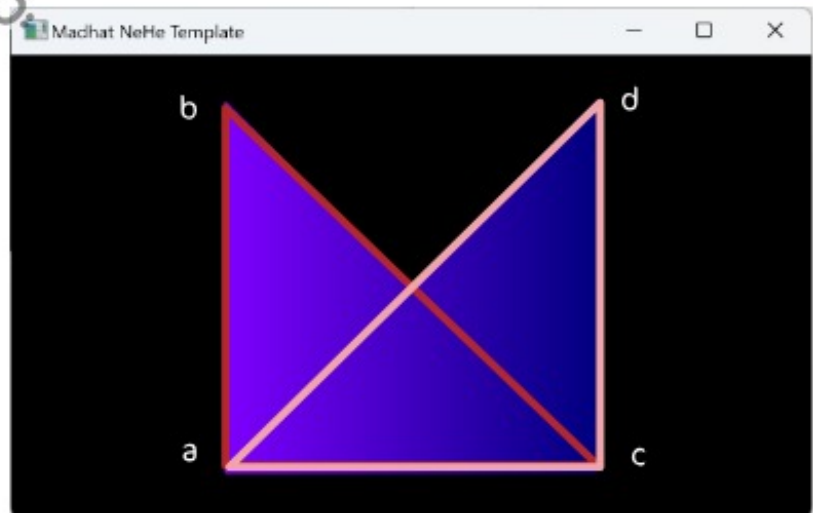
يرسم مجموعة من المثلثات المتصلة من نقطة مركزية. كل مثلث يتكون من نقطة مركزية ونقطتين ضافيتين. فمثلاً عند إضافة النقاط a, b, c, d سيقوم باعتبار أن أول نقطة a هي المركز وسيشكل المثلثات ذات الرؤوس: abc, acd .

مثال:

```
glBegin(GL_TRIANGLE_FAN);
glColor3f(0.5f, 0.0f, 1.0f);
// اللون البنفسجي
glVertex3f(-1.0f, -1.0f, -3.0f);
// المركز
glVertex3f(-1.0f, 1.0f, -3.0f);
// النقطة 1
glColor3f(0.0f, 0.0f, 0.5f);
// اللون الأزرق
glVertex3f(1.0f, -1.0f, -3.0f);
// النقطة 2
glVertex3f(1.0f, 1.0f, -3.0f);
// النقطة 3
glEnd();
```



توضيح:



ملاحظة: عند رسم مثلث مثلاً وتحديد كل نقطة من نقاطه بلون مختلف عن الآخر سيتلون كل رأس من رؤوس المثلث بهذا اللون أما في المنتصف سيتم دمج هذه الألوان سوياً، كما تم دمج اللونين الأزرق والبنفسجي في المثال السابق.



3. GL_TRIANGLE_STRIP

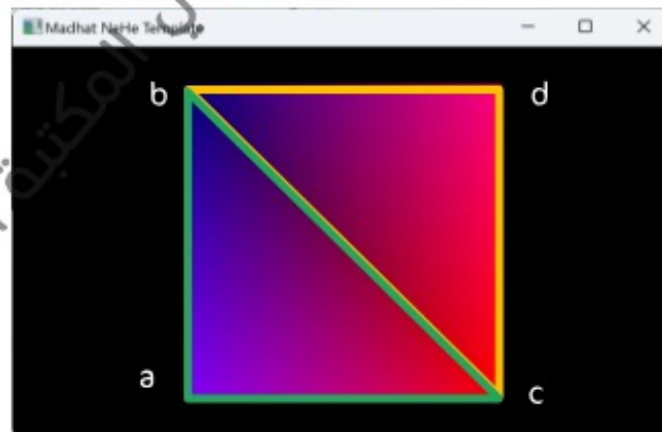
يرسم مثلثات متتالية باستخدام النقاط السابقة والجديدة لتشكيل مثلثات متصلة. حيث يشكل مثلثاً جديداً من كل نقطة جديدة مع النقطتين السابقتين لها فمثلاً عند إضافة النقاط a, b, c, d سيشكل المثلثات ذات الرؤوس: abc, bcd

مثال:

```
glBegin(GL_TRIANGLE_STRIP);
glColor3f(0.5f, 0.0f, 1.0f);
// اللون البنفسجي
glVertex3f(-1.0f, -1.0f, -3.0f);
// النقطة 1
glColor3f(0.0f, 0.0f, 0.5f);
// اللون الأزرق
glVertex3f(-1.0f, 1.0f, -3.0f);
// النقطة 2
glColor3f(1.0f, 0.0f, 0.0f);
// اللون الأحمر
glVertex3f(1.0f, -1.0f, -3.0f);
// النقطة 3
glColor3f(1.0f, 0.0f, 0.5f);
// اللون الذهبي
glVertex3f(1.0f, 1.0f, -3.0f);
// النقطة 4
glEnd();
```



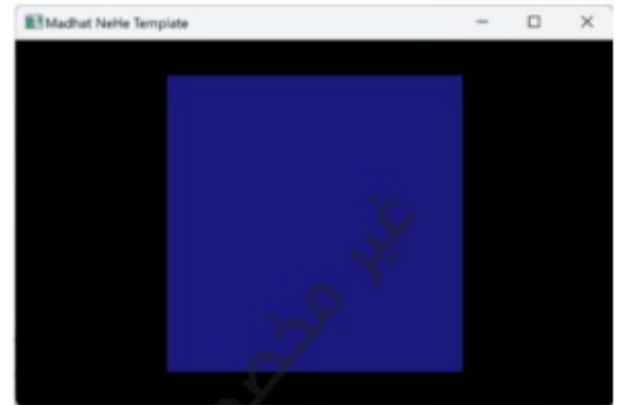
توضيح:



4. GL_QUADS

الوصف: تستخدم لرسم الأشكال الرباعية مثل المربعات أو المستطيلات أو أي شكل ينتج عن 4 نقط.
الخصائص: يتطلب رسم شكل رباعي أربع نقاط. يمكن استخدامه لرسم الأسطح المسطحة، ويكون معبئاً من الداخل بشكل افتراضي.

```
glBegin(GL_QUADS);
glColor3f(0.1f, 0.1f, 0.5f);
// اللون الأزرق
glVertex3f(1.0f, 1.0f, -3.0f);
glVertex3f(1.0f, -1.0f, -3.0f);
glVertex3f(-1.0f, -1.0f, -3.0f);
glVertex3f(-1.0f, 1.0f, -3.0f);
glEnd();
```



يتم تجاهل النقط الأخيرة إذا لم يكن عدد النقط من مضاعفات الـ 4.

أنماط مختلفة:

1. GL_QUAD_STRIP

يرسم سلسلة من المربعات المتصلة بحيث تشكل سلسلة من الأشكال الرباعية، حيث يأخذ آخر نقطتين مضافتين مع النقطتين السابقتين لهما ويشكل منهما رباعي، فمثلاً عند إضافة النقاط a, b, c, d, e, f, g, h سيرسم الرباعيات: abcd, cdef, efgh.

مثال:

```
glBegin(GL_QUAD_STRIP);
glColor3f(1.0f, 0.0f, 0.0f);
// اللون الأحمر
glVertex3f(1.0f, 1.0f, -3.0f);
glVertex3f(-1.0f, 1.0f, -3.0f);
glVertex3f(1.0f, -1.0f, -3.0f);
glVertex3f(-1.0f, -1.0f, -3.0f);
// رباعي أول
glColor3f(1.0f, 1.0f, 0.0f);
// اللون الأصفر
glVertex3f(0.0f, 1.0f, -3.0f);
glVertex3f(-2.0f, 1.0f, -3.0f);
// رباعي ثاني
glVertex3f(0.0f, -0.5f, -3.0f);
glVertex3f(-2.0f, -0.5f, -3.0f);
// رباعي ثالث
glEnd();
```

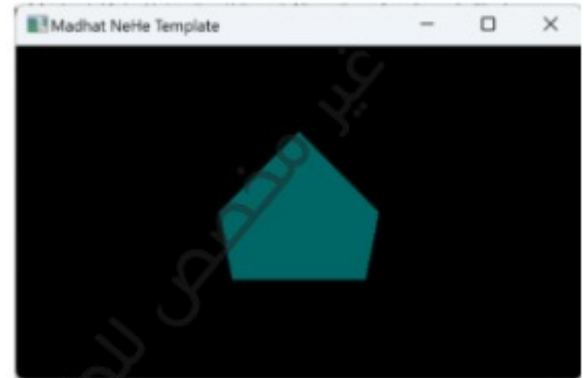


الوصف: تستخدم لرسم مضلعات ذات عدد غير محدد من الرؤوس.

الخصائص: يمكن رسم مضلعات ذات أكثر من رؤوس، وتكون ممثلة بشكل افتراضي (مظلمة).

مثال:

```
glBegin(GL_POLYGON);
glColor3f(0.0f, 0.4f, 0.4f);
glVertex3f(-0.5f, -0.5f, -3.0f);
glVertex3f(0.5f, -0.5f, -3.0f);
glVertex3f(0.6f, 0.0f, -3.0f);
glVertex3f(0.0f, 0.6f, -3.0f);
glVertex3f(-0.6f, 0.0f, -3.0f);
glEnd();
```



Winding في OpenGL

winding هو مصطلح يستخدم في OpenGL لوصف ترتيب واتجاه تحديد الرؤوس (vertices) عند رسم الأشكال. يعتبر هذا المفهوم مهماً جداً في رسم الأشكال، لأنه يؤثر على كيفية معالجة الأوجه الأمامية والخلفية للأشكال وكيفية ملئها بالألوان.

أهمية الـ winding

1. تحديد الوجه الأمامي والخلفي:

عندما نقوم برسم شكل مثل مثلث، يتم تحديد أي وجه هو الوجه الأمامي (الذي سيكون مرئياً) بناءً على ترتيب الرؤوس. حيث أننا إذا قمنا بتحديد رؤوس المثلث عكس عقارب الساعة فسيكون الدوران موجباً وسيظهر لنا الوجه الأمامي للمثلث، أما في حال حددنا الرؤوس مع عقارب الساعة فسيكون الدوران سالباً وسيظهر لنا الوجه الخلفي للمثلث.

2. تأثيره على ملء الأشكال:

يمكن التحكم في كيفية تحديد الجهة الأمامية والخلفية للمضلعات باستخدام `glPolygonMode`، وعند استخدامها يمكنك تحديد كيف سترسم الأشكال (سواء كأوجه مملوءة أو كحدود فقط).

```
glPolygonMode(face, mode);
```

■ Face: يُحدد أي وجه سيتم تطبيق النمط عليه، ويمكن أن يكون أحد القيم التالية:

GL_FRONT: لتحديد الوجوه الأمامية. 🔴

GL_BACK: لتحديد الوجوه الخلفية. 🔴

GL_FRONT_AND_BACK: لتحديد كلا الوجوه الأمامية والخلفية. 🔴

Mode: يُحدد كيف سيتم رسم الوجه، ويمكن أن يكون أحد القيم التالية:

GL_FILL: لملء الوجه باللون.

GL_LINE: لرسم خطوط خارجية فقط (حدود).

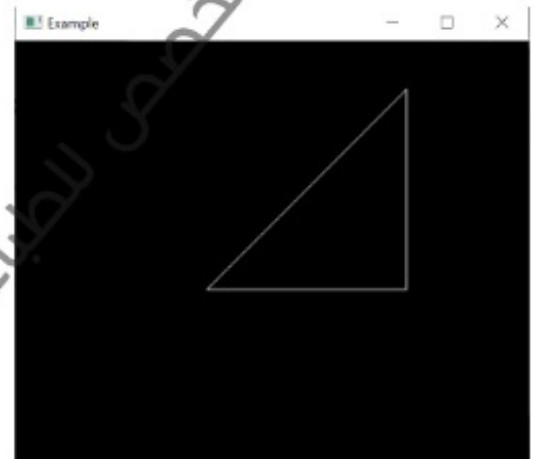
GL_POINT: لرسم النقاط فقط.

عند استدعاء `glPolygonMode(GL_FRONT, GL_FILL)`، سيتم ملء الوجه الأمامي بلون معين.

إذا أردت رسم الحدود فقط دون ملء، يمكنك استخدام `glPolygonMode(GL_FRONT, GL_LINE)`.

ملاحظة: الحالة الافتراضية هي `GL_FILL`.

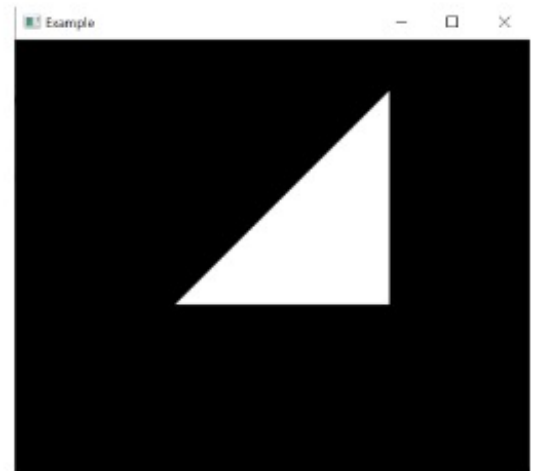
```
glPolygonMode(GL_FRONT, GL_LINE);
glBegin(GL_TRIANGLES);
    glVertex3f(0.0f, 0.0f, -3.0f); // V0
    glVertex3f(1.0f, 0.0f, -3.0f); // V1
    glVertex3f(1.0f, 1.0f, -3.0f); // V2
glEnd();
```



نلاحظ أن التغييرات ظهرت لأننا رسمنا العقد بعكس اتجاه عقارب الساعة فظهر لنا الوجه الأمامي الذي قمنا بالتعديل عليه.

للتأكد من أن الوجه الخلفي لم يتم التعديل عليه وبقي بوضعه الافتراضي يمكن أن نعيد رسم العقد ولكن باتجاه عقارب الساعة:

```
glPolygonMode(GL_FRONT, GL_LINE);
glBegin(GL_TRIANGLES);
    glVertex3f(1.0f, 0.0f, -3.0f); // V1
    glVertex3f(0.0f, 0.0f, -3.0f); // V0
    glVertex3f(1.0f, 1.0f, -3.0f); // V2
glEnd();
```



BACK

يبقى على الحالة الافتراضية

FRONT

يتأثر بالتغيرات الحاصلة في الكود

3. مستقبلاً سنلاحظ أن ال FRONT هو فقط الذي يتأثر بالإضاءة.



If you look at what you have in life,
you'll always have more. If you look at
what you don't have in life, you'll
never have enough.